

1 Actor Network in SAC

In Soft Actor-Critic (SAC), the actor network outputs both the mean μ and the log standard deviation $\log \sigma$ to parameterize a Gaussian policy:

$$\pi(a|s) = \mathcal{N}(\mu(s), \sigma^2(s)).$$

This design enables stochastic policy learning, which is crucial for entropy maximization in SAC.

1. Why output mean and log std?

Outputting μ and $\log \sigma$ allows us to:

- Construct a Gaussian distribution from which actions can be sampled.
- Ensure $\sigma > 0$ by exponentiating $\log \sigma$.
- Learn a stochastic policy to support entropy regularization:

$$\mathbb{E}_{a \sim \pi(\cdot|s)} [Q(s, a) + \alpha \mathcal{H}(\pi(\cdot|s))].$$

- Keep the action bounded in $[-1, 1]$ using a tanh squashing function:

$$a = \tanh(z), \quad z \sim \mathcal{N}(\mu, \sigma^2).$$

2. Reparameterization Trick

To make the action sampling differentiable, the reparameterization trick is used:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

implemented via `dist.rsamle()` in PyTorch. This allows gradients to propagate through the sampling process.

3. Correcting the log probability

Due to the tanh squashing, the density of the transformed action is not Gaussian. Hence, the log-probability must be adjusted using the change-of-variables formula:

$$\log \pi(a|s) = \log \mathcal{N}(z|\mu, \sigma^2) - \log \left| \frac{d \tanh(z)}{dz} \right|.$$

In implementation:

```
log_prob = dist.log_prob(z) - torch.log(1 - action.pow(2) + 1e-7)
```

This correction ensures accurate estimation of policy entropy and gradients, as detailed in Appendix C of the SAC paper.

2 Soft Q Network in SAC

1. Why Use Twin Q Networks?

SAC adopts two separate Q-networks, Q_1 and Q_2 , to mitigate the overestimation bias that commonly occurs in value-based methods. When estimating targets like

$$r + \gamma \max_{a'} Q(s', a'),$$

using the same network for both selecting and evaluating a' can lead to overly optimistic value estimates.

To address this, SAC uses the **Clipped Double Q-learning** trick, which first appeared in TD3. Specifically, the target is computed using:

$$\min(Q_1(s, a), Q_2(s, a)),$$

thus providing a more conservative and stable estimate.

2. Q-function Loss

The target for the soft Q-function is defined by:

$$y^Q = r + \gamma(1 - d) \cdot V(s'),$$

where d is the done flag. Each Q-network is updated by minimizing the mean squared error:

$$\mathcal{L}_{Q_i} = \mathbb{E} \left[(Q_i(s, a) - y^Q)^2 \right], \quad i = 1, 2.$$

This twin Q-learning setup helps SAC achieve more robust value estimation and faster convergence during training.

3 Entropy Temperature Tuning in SAC

1. Motivation

In the original SAC, the temperature parameter α that controls the entropy regularization strength is manually tuned. However, this fixed α may lead to suboptimal behavior:

- Too large $\alpha \Rightarrow$ overly stochastic policy (poor exploitation)
- Too small $\alpha \Rightarrow$ deterministic policy (poor exploration)

2. Automatic Entropy Adjustment

To address this, the improved SAC introduces automatic entropy tuning by treating α as a learnable parameter.

The objective is to minimize:

$$\mathcal{L}(\alpha) = \mathbb{E}_{a \sim \pi} [-\alpha \log \pi(a|s) - \alpha \cdot \mathcal{H}_{\text{target}}],$$

which has the gradient:

$$\nabla_{\alpha} \mathcal{L} = -\log \pi(a|s) - \mathcal{H}_{\text{target}}.$$

In practice, the loss is implemented as:

```
alpha_loss = (-log_alpha.exp() * (log_prob + target_entropy).detach()).mean()
```

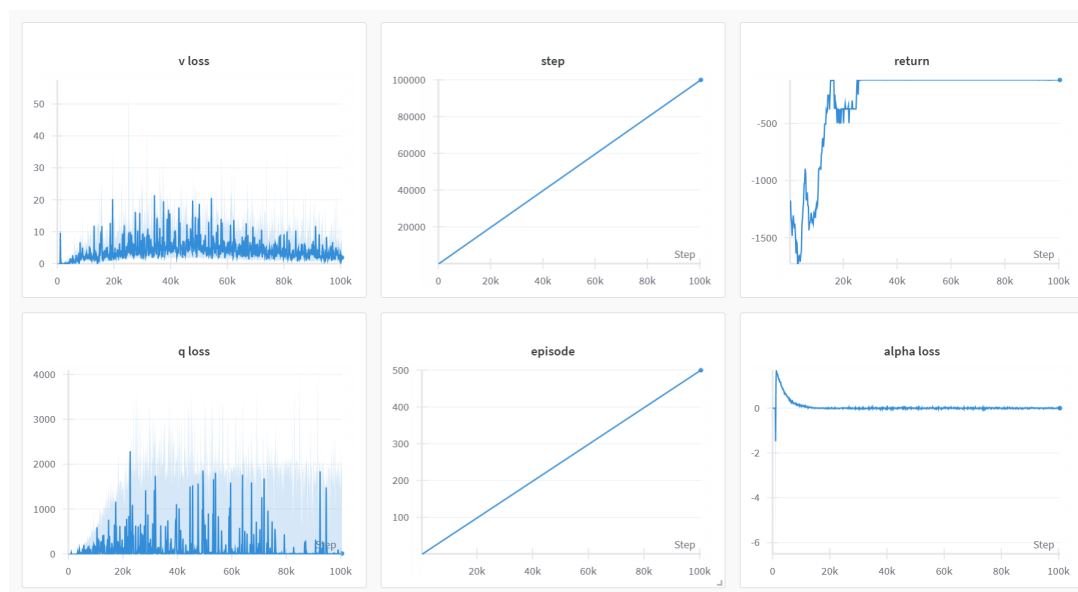
This allows α to adaptively increase or decrease, encouraging the policy to maintain a target level of entropy during training.

4 SAC on Pendulum-v1: Experimental Results and Hyperparameters

Training Performance

- **Return (episodic reward):** The return steadily improved from around -1600 to approximately -180 within the first 25k steps and remained stable thereafter, indicating convergence to a near-optimal policy.
- **Q Loss:** The Q-function loss initially fluctuated due to bootstrap targets, peaked early, then gradually stabilized. Occasional spikes are expected due to stochastic sampling.
- **V Loss:** The value function loss followed a similar trend—gradual rise in early training followed by convergence and smoothing after 40k steps.
- **Alpha Loss:** The automatic entropy tuning mechanism showed strong early adaptation. The alpha loss decreased quickly and converged toward zero.
- **Episodes and Steps:** Approximately 500 episodes were completed in 100,000 steps, implying an average episode length of about 200 steps.

W&B Snapshot



The plots above were generated using Weights & Biases. They show the evolution of the policy's return and each loss component across training.

Hyperparameters

Parameter	Value
Environment	Pendulum-v1
Learning Rate	3×10^{-4}
Discount Factor γ	0.99
Soft Update Rate τ	0.005
Batch Size	128
Initial Random Steps	1000
Replay Buffer Size	100,000
Policy Update Frequency	2
Hidden Layer Size	128×2 (ReLU)
Training Steps	100,000
Target Entropy	-1 (automatically inferred from action dim)
Optimizer	Adam

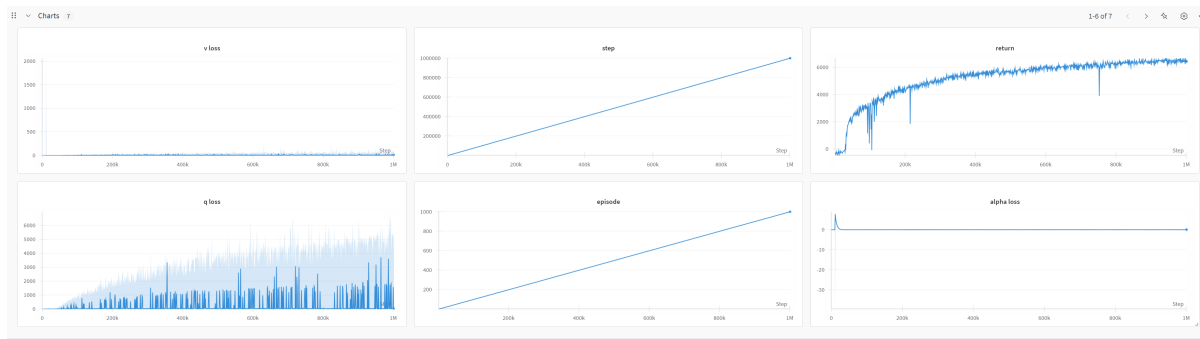
表 1: Hyperparameters used in SAC training on Pendulum-v1

5 (e) SAC on HalfCheetah-v5: Experimental Results and Hyperparameters

We applied our Soft Actor-Critic (SAC) implementation to the more complex MuJoCo environment **HalfCheetah-v5**. The training was conducted for 1 million interaction steps using GPU acceleration. The following summarizes the experimental results and setup:

Training Performance

- **Return:** The episodic return quickly rose from below 1000 to over 6000 and continued to increase steadily. The final return plateaued around 6500–7000, indicating successful locomotion learning.
- **V Loss:** Value loss spiked early but then decreased and stabilized, showing effective convergence of the soft value function.
- **Q Loss:** Q-function loss steadily increased with training but remained within a bounded range, typical of soft Q-learning with bootstrapped targets.
- **Alpha Loss:** The temperature parameter α stabilized quickly, and its loss approached zero, confirming convergence of the entropy regularization process.
- **Episodes:** Approximately 1000 episodes were completed during the 1 million steps, implying an average episode length close to the environment limit of 1000 steps.



Hyperparameters

Parameter	Value
Environment	HalfCheetah-v5 (MuJoCo)
Learning Rate	3×10^{-4}
Discount Factor γ	0.99
Soft Update Rate τ	0.005
Batch Size	256
Initial Random Steps	10000
Replay Buffer Size	1,000,000
Policy Update Frequency	1
Hidden Layer Size	128×2 (ReLU)
Training Steps	1,000,000
Target Entropy	$-n$ (where n is the action dimension)
Optimizer	Adam

表 2: Hyperparameters used in SAC training on HalfCheetah-v5

Conclusion

The SAC algorithm demonstrated successful learning on the HalfCheetah locomotion task, achieving the target performance threshold of 7000 within the allocated 1 million steps. The result is consistent with the expectations reported in the original SAC paper.